

Binairo 2026 — draft project report

Draft; not submission-ready. This short report describes the frozen implementation contract and the verified C stage. Native MATLAB/LabVIEW results and the final current submission rules must be incorporated before final submission. This draft is exported separately within the inherited 1–3-page target (5-page maximum). Detailed construction, operator and test guides remain separate.

Authors: the three team members' actual names are pending; no example names have been substituted. Local C test platform: macOS ARM64. Compiler: Apple clang 21.0.0, target arm64-apple-darwin25.6.0. MATLAB/LabVIEW versions and final target computer: pending human native evidence. AI assistance is disclosed; complete available conversation PDFs and external-source references are prepared separately.

Files and dataflow

BinairoSolver.vi is the entry point. Native LabVIEW reads and displays a PNG, renders transparency against white with the native 128 alpha-mask threshold, converts to black/white, locates the black-pixel bounds, detects the square even grid and crops each cell interior through ComputeRowsCols.vi and ComputeCellRect.vi. Scanlines are 10 pixels inside the detected top/left border. It writes Cell.bin, invokes the target-platform C executable OCR / OCR.exe, then validates and reads CellValue.txt for each cell. It accumulates numbers 0/1 and NaN for blanks into a MATLAB matrix and generates solve.m with the user-selected source and options. A fully specified synchronous native System Exec adapter launches MATLAB; the supplied MP_LaunchMatlabScript4.vi is retained as a course-compatible route after its real interface is manually verified.

SolveBinairo.m and helpers solve the matrix. DisplayBinairo.m writes the PDF; DispBinairo.m provides naming compatibility. RunBinairo.m handles the generated script's workflow and diagnostic files. All runtime functions, VIs, executable and generated files are placed in one runtime folder, with paths derived from the VI/script location. Input puzzles may be elsewhere. There are no machine-specific personal paths in runtime code. Development/test helpers are separate from assessed runtime functionality.

Algorithms and interfaces

C reads width and height as little-endian U32, followed by exactly width×height byte pixels in row-major order, 0 white and 1 black. It validates the header, values and size, allocates with malloc, reads the payload in one block and releases resources on every exit. No VLA is used. Genuine helper functions access pixels and perform recognition. Empty detection runs first using the white percentage. For each supplied packed 32×32 digit bitmap, unsigned bit operations extract pixels; every fitting translation compares both black and white pixels. The best raw similarity is retained for each digit.

The provisional selection rule keeps scores at least their digit threshold and selects the greatest score-minus-threshold; ties prefer higher raw score then 0. Blank requires a strictly greater white percentage than its threshold. These rules reconcile conflicting current descriptions explicitly in the decision log. CLI arguments are cell path and three thresholds, empty/0/1. Output examples are d:'0',93.7500% and d:' ',100.0000%, each with newline; the blank token and four-decimal grammar are project conventions awaiting confirmation.

MATLAB preserves original clues, checks no consecutive triples, symbol-count limits and uniqueness of completed rows/columns, propagates forced values to a fixed point, then recursively guesses 0 followed by 1 at the first remaining empty cell in column-major order. Vectorized checks and recursive propagation avoid explicit for/while in assessed functions while preserving the required direct-deduction/backtracking method. The first valid solution is returned; uniqueness is not claimed.

The PDF shows original clues in black bold, inferred values in blue bold, all grid lines, source filename at upper left and timestamp at upper right. An infeasible attempted puzzle displays == Error == centrally. Under the provisional option semantics, disabling solving renders original clues; disabling Show pdf suppresses opening, not file creation.

Errors and verification

C returns 0 on success; failures use stderr OCR E<number>: description and exit 2 for arguments/path, 3 for format/dimensions including premature EOF, 4 for actual stream/open/allocation failure, 5 for no recognition, 6 for output I/O. It removes obsolete output before processing a valid input path and avoids partial result publication. LabVIEW must invalidate old results and read only after a clear error cluster, zero return code and empty stderr; malformed/missing result is an error. Script failures record status, full error details and a log. Native process/launcher/status propagation still needs execution evidence.

Measured local C results: 49 unit checks and 69 CLI integration cases passed, each in normal and AddressSanitizer/UndefinedBehaviorSanitizer builds. Tests include the supplied cells, nonsquare cells, final offsets, thresholds/ties, malformed/truncated/excess payload, invalid pixels, no-match, output failure and paths with spaces/apostrophes. The CLI oracle uses independent integer-bitset comparisons. Two additional empty-output-directory regression checks passed in both builds. An initial incompatible Python 3.9 run was superseded by a successful Python >=3.10 run; the failure remains disclosed. A separate Python Pillow+C surrogate matches supplied 6x6/8x8/4x4 matrices and rejects the 6-row/5-column grid; it does not execute LabVIEW. Not every OS fault was injected. Windows compilation/execution and all native MATLAB, LabVIEW, launcher, PDF and end-to-end tests remain pending.

Remaining official points and references

Current technical sources are P00.PPI_Projet.2026.2.pdf physical pp7–23 (C),24–27 (LabVIEW),28–36 (MATLAB/integration), and the specific OCR/LabVIEW/MATLAB exercises. General inherited grading/documentation comes from P0.PPI_Projet.25.r6.pdf pp38–43,48–54 and its rev.2 addendum pp4–7,14,17–18, as confirmed in the user's instructions. Exact references are in the source log.

The current Binairo connector, final error/submission rules, official thresholds, margin/raw-score conflict, dimension bound, blank token and naming/options remain documented questions. Cell bounds 10..256, border-free defaults 98/90/90 and other unresolved interface details are provisional. No native success, completed VI, complete transcript history or grade is claimed without evidence.